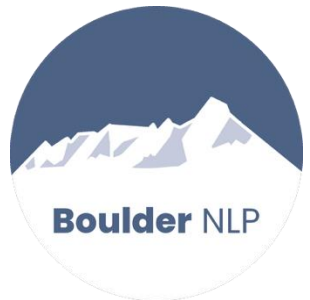


Natural Language Processing

Jim Martin -- Lecture 6
CSCI 5832



Today

- Finish motivation for sigmoid
 - Logits
- Learning for Logistic Regression
 - Chapter 4

Natural Language Processing

Go to

join.iClicker.com
YXSF



Share this join link with students

<https://join.iclicker.com/YXSF>

Copy

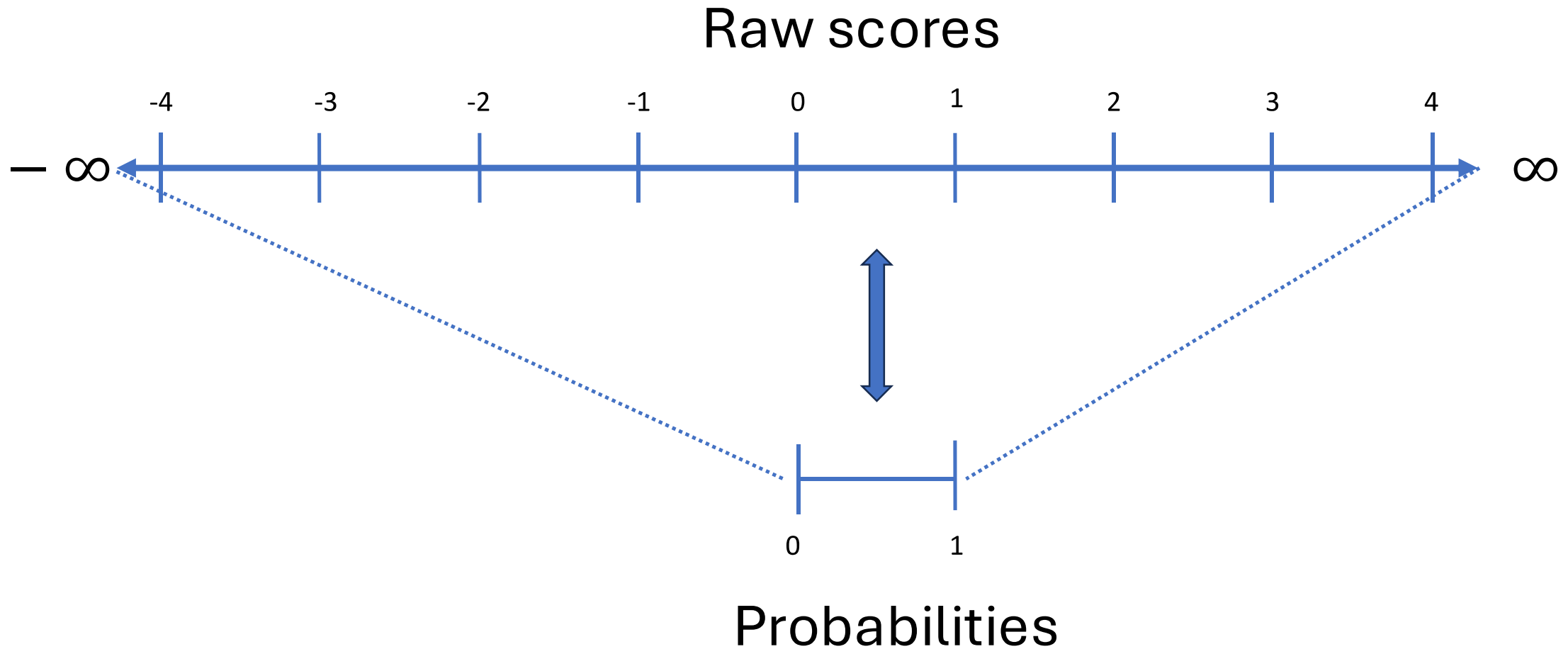
Motivation

Before diving in to learning, let's review some details with our scoring function.

$$z = \mathbf{w}^T \mathbf{x}$$

- We haven't restricted our weights , the bias term, or the feature values to anything other than real numbers.
- Therefore, the possible range of scores (z) can extend from $-\infty$ to ∞ . That is, the possible scores lie anywhere along the real number line.

To Infinity and Beyond



Step 1: From Probabilities to Odds

Without any loss of information, in the binary case, we can express probabilities as simple odds: the ratio of one possibility to the other.

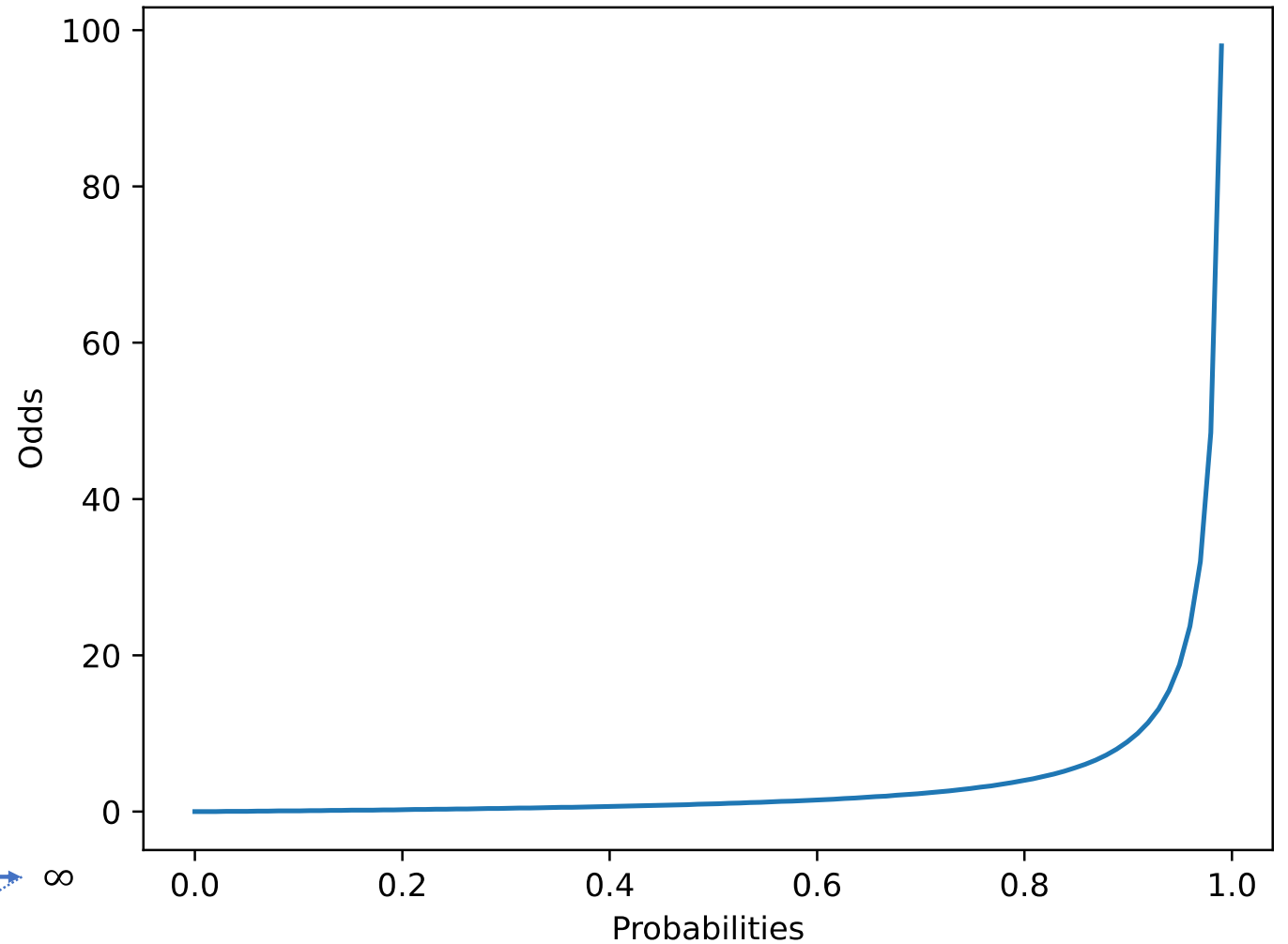
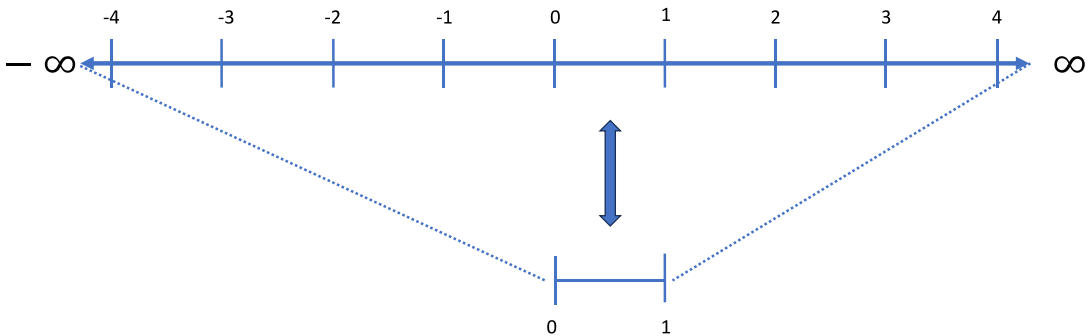
.95	19 to 1	19
.8	4 to 1	4
.75	3 to 1	3
.5	1 to 1	1
.25	1 to 3	.3333
.2	1 to 4	.25
.05	1 to 19	.0526

$$\frac{p}{1 - p}$$

For example, if the probability of some (binary) event is 0.75 then the odds are 0.75/0.25 or 3/1

Step 1: Probabilities to Odds

While probabilities
range from 0 to 1,
odds range from 0 to
infinity.



Step 1: Probabilities to Odds

- Moving on we can observe that there's an annoying asymmetry to our odds values.
- 19 to 1 and 1 to 19 really should be related somehow, but they're not obviously --- we have a 19 and a .0526.

<u>.95</u>	<u>19 to 1</u>	<u>19</u>
.8	4 to 1	4
.75	3 to 1	3
.5	1 to 1	1
.25	1 to 3	0.3333
.2	1 to 4	0.25
<u>.05</u>	<u>1 to 19</u>	<u>0.0526</u>

Step 2: Odds to Log Odds

- Let's take the log of the odds!

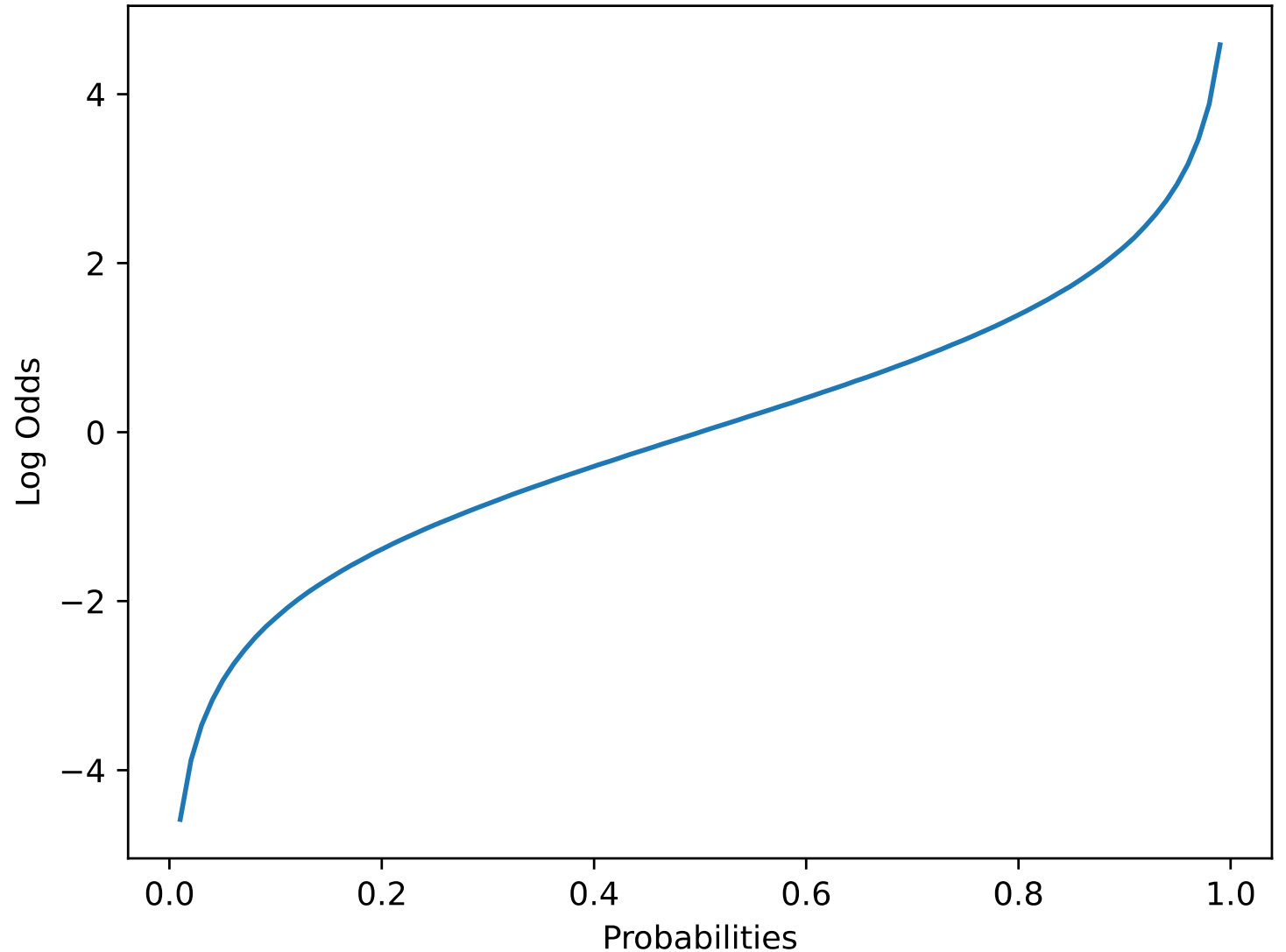
$$\ln \left(\frac{p}{1-p} \right)$$

.95	19 to 1	19	<u>2.944</u>
.8	4 to 1	4	1.38
.75	3 to 1	3	1.098
.5	1 to 1	1	0
.25	1 to 3	.3333	-1.098
.2	1 to 4	.25	-1.38
.05	1 to 19	.0526	<u>-2.944</u>

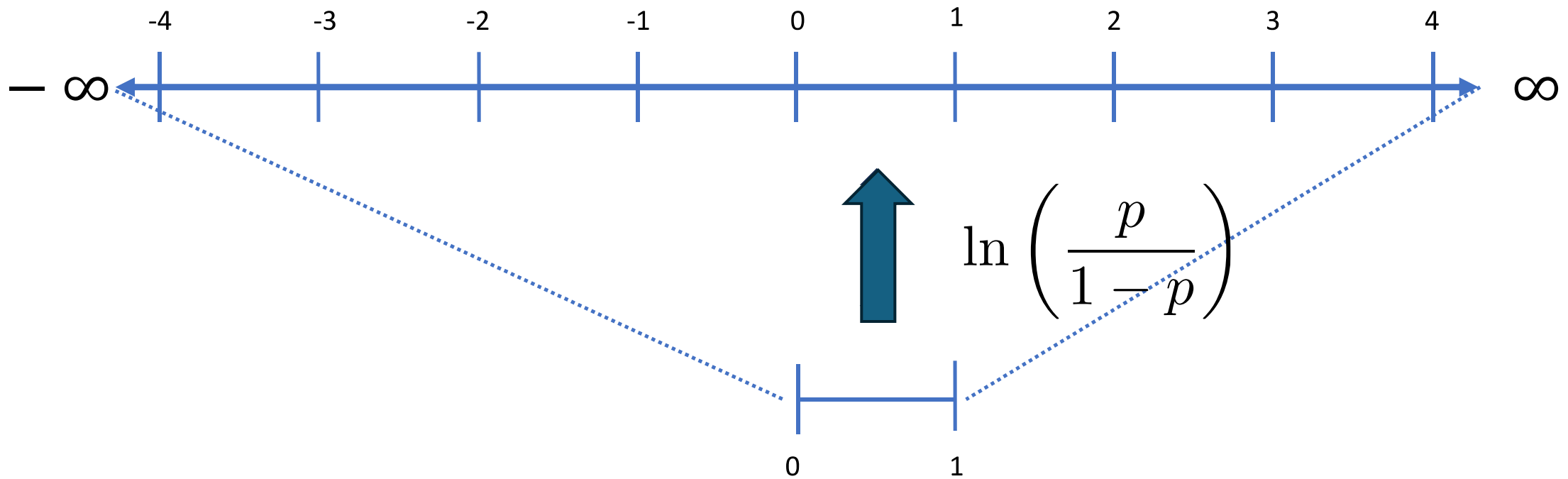
- Symmetry!
- And we've found a way to map probabilities to the real number line. Centered around 0 (no information to distinguish).

Step 2: Odds to Log Odds

- While probabilities range from 0 to 1, log odds range from $-\infty$ to ∞ .



Reals to Probabilities



Logit

The log of the odds of the probability of an event is called the logit.

$$\textit{logit} = \ln \left(\frac{p}{1 - p} \right)$$

- The scores z from our $(w^T x)$ computation are logits. During learning we will be searching for weights w that produce the right logits.
- So, now we know how take a probability and produce its logit. How do we go the other way? Given a logit, produce a probability.
- We've already seen the answer, but let's do some algebra to be sure.

$$z = \ln \left(\frac{p}{1-p} \right)$$

Let's call our logit z .

$$e^z = \frac{p}{1-p}$$

Exponentiate both sides to rid us of that pesky log.

$$e^z(1-p) = p$$

Some algebra to isolate p .

$$e^z - pe^z = p$$

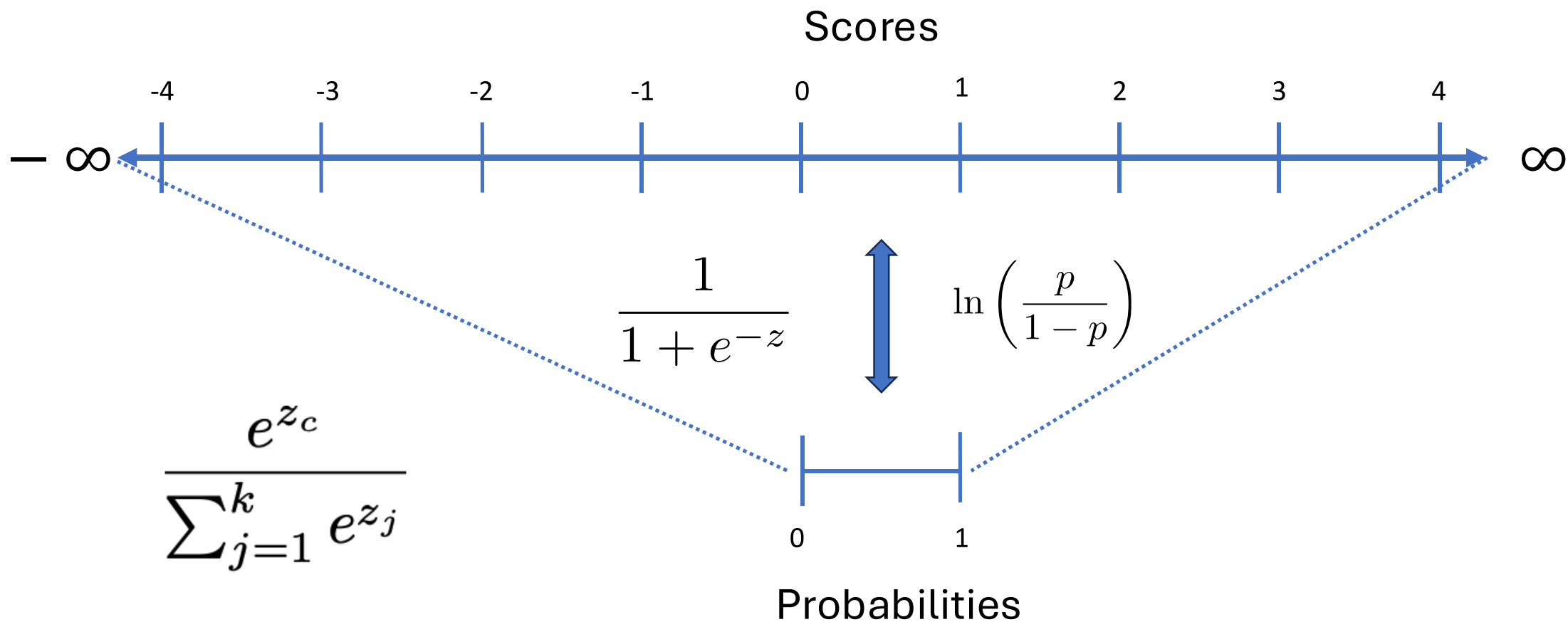
$$e^z = p + pe^z$$

And voila! The sigmoid – p in terms of z .

$$e^z = p(1 + e^z)$$

$$p = \frac{e^z}{1 + e^z} = \frac{1}{1 + \frac{1}{e^z}} = \frac{1}{1 + e^{-z}}$$

To Infinity and Beyond



Take Aways

- The logit is the log of the odds of some event. Where the odds is the ratio of the probability of an event being the case to the probability of it not being the case.
- The raw scores from our linear classifier are logits. Logits will play a central role later in the course as we move from simple models to deep learning.
- Our next big task is to come up with a learning scheme to acquire the proper weights w , so that our scores represent logits appropriate to our task.

Clicker Question

Assume a binary classifier has assigned a probability of 0.5 to an item.

What is the logit for the item?

Learning

- So... where do the weights come from?

Learning

- So... where do those weights come from?
- We'll learn the weights from a training set of documents already labeled with the right answer.
 - We'll use 1 and 0 as labels to stand for the right answer (positive or negative)
 - The system answers will be between 0 and 1

Weights

- To learn the weights, we need some measure of how well (or badly) we're doing with a current set of weights.
- We'll call that measure a *Loss Function*
- The lower the loss the better we're doing
 - We want to minimize the loss

Loss Functions

What we want is a function that tells us how well our model is doing.

And does it in a way that can be used to guide the training process

Loss Functions

There are lots of ways to evaluate how well we're doing on a test/validation set.

- Accuracy, F1, Precision, recall.
 - All discrete
- How close are our answers are to the correct answers?
 - What does close even mean?
- What's the nature of the overall loss over the entire training/test data

Cross-Entropy Loss

Let's start with the following notion

the probability assigned to the correct class by a model

- In the binary case, the correct answer is always either 1 or 0.
- The model output is always a number between 0 and 1. That represents the model's prediction of the probability that the item belongs to the positive class.
 - By convention, the probability given by the model is the probability that the document belongs to class 1 (or the positive class)

Cross Entropy Loss

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

y is the correct answer.

y_hat is the system answer.

Logs

Probabilities are the opposite of what we want for a loss.

- We want bad performance to have high loss, and good performance to have low loss.
- So, we'll take the negative of the log of the probability assigned to the correct answer as the loss

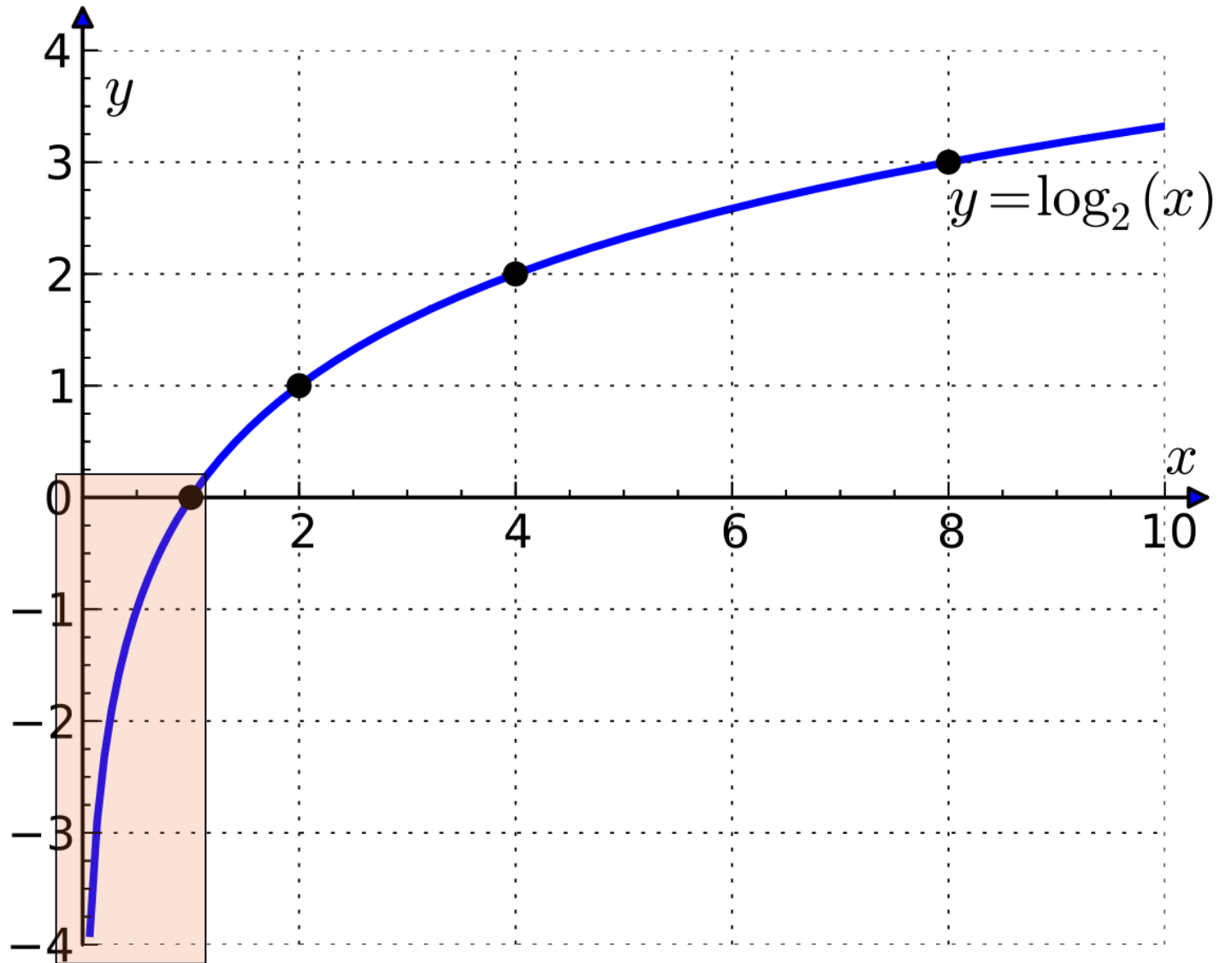
Logs

Log of a value between 0 and 1 ranges from -infinity to 0.

Taking the negative of that flips it over the axis, so it ranges from infinity to 0.

Low prob $\rightarrow$ large value

High prob $\rightarrow$ small value



Cross Entropy Loss

$$p(y|x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

$$\begin{aligned}\log p(y|x) &= \log [\hat{y}^y (1 - \hat{y})^{1-y}] \\ &= y \log \hat{y} + (1 - y) \log (1 - \hat{y})\end{aligned}$$

$$L_{CE}(w, b) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))]$$

The negative of the log probability assigned to the correct class.

Learning

Find the weights that minimize the average loss across an entire training set.

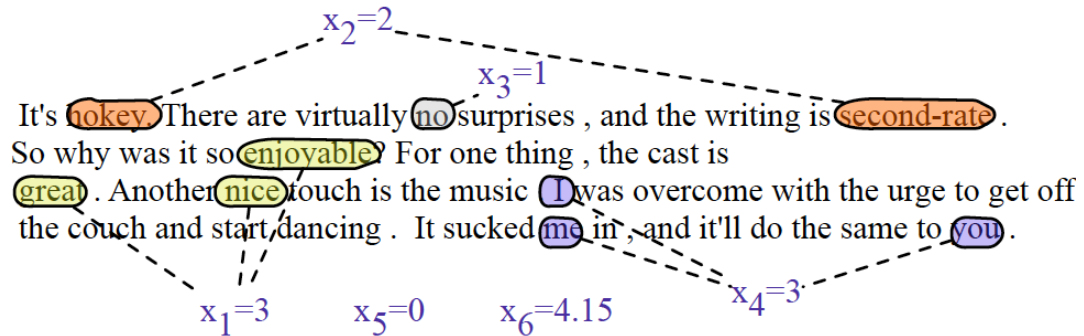
$$\begin{aligned} \text{Cost}(w, b) &= \frac{1}{m} \sum_{i=1}^m L_{CE}(\hat{y}^{(i)}, y^{(i)}) \\ &= -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log \sigma(w \cdot x^{(i)} + b) + (1 - y^{(i)}) \log (1 - \sigma(w \cdot x^{(i)} + b)) \end{aligned}$$

Learning

We'll do this by starting with a random set of weights and then iteratively updating those weights to lower this cost.

$$\begin{aligned} \text{Cost}(w, b) &= \frac{1}{m} \sum_{i=1}^m L_{CE}(\hat{y}^{(i)}, y^{(i)}) \\ &= -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log \sigma(w \cdot x^{(i)} + b) + (1 - y^{(i)}) \log (1 - \sigma(w \cdot x^{(i)} + b)) \end{aligned}$$

Scoring



 **[3,2,1,3,0,4.15]**

$$\begin{aligned} p(+|x) &= P(Y = 1|x) = \sigma(w \cdot x + b) \\ &= \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.15] + 0.1) \\ &= \sigma(.805) \\ &= 0.69 \end{aligned} \tag{5.6}$$

$$\begin{aligned} p(-|x) &= P(Y = 0|x) = 1 - \sigma(w \cdot x + b) \\ &= 0.31 \end{aligned}$$

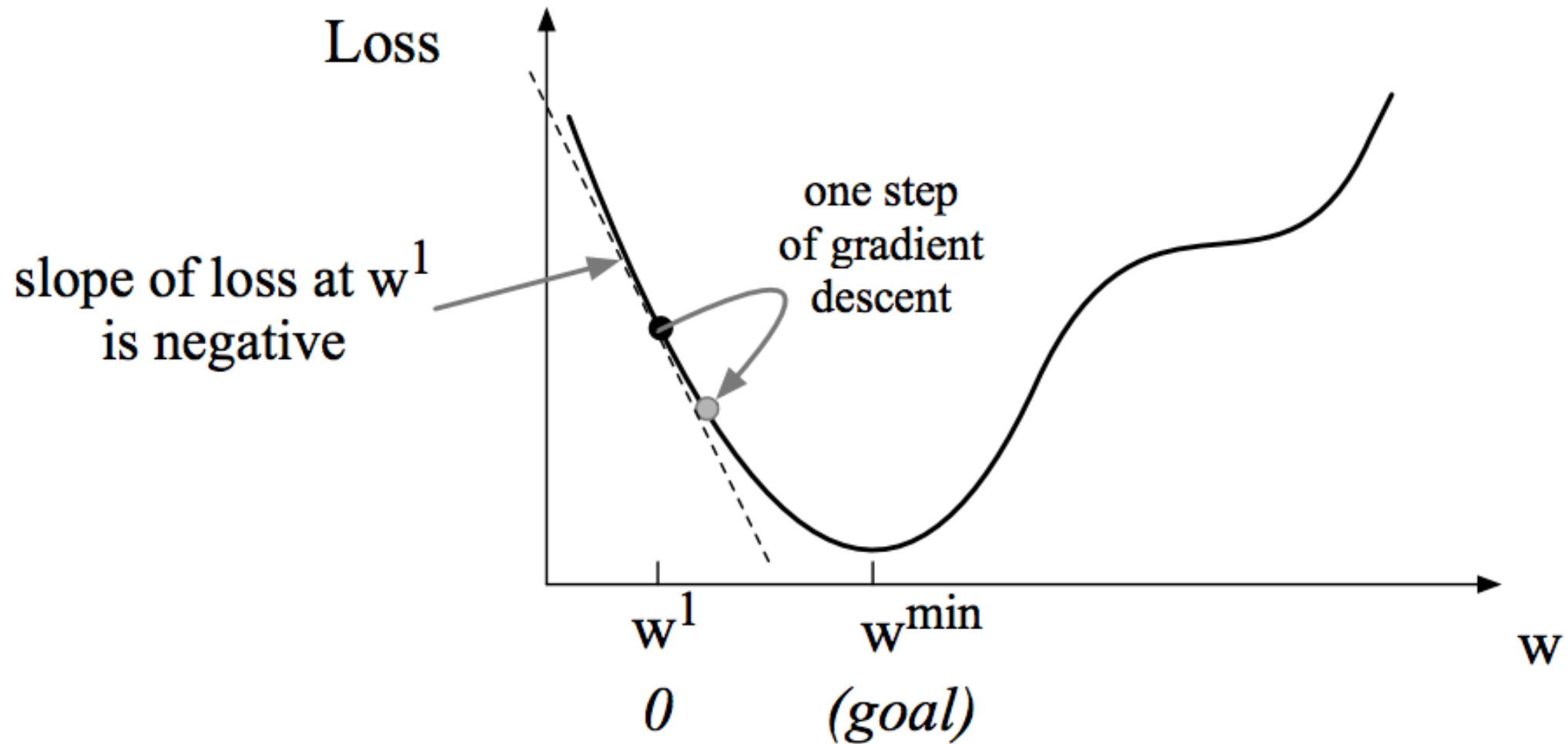
Learning

$$w_{t+1} = w_t - \mu \Delta$$

Derivatives

- Fortunately, multivariate calculus tells us how to do that.
- The derivative of the loss function with respect to the weights tells us the direction and magnitude of change we should make to each of the weights.

Single Weight



Partial Derivative

- Of course, in real applications we have many features/weights not just one.
- So we need the vector of the partial derivatives of the loss wrt the weights
 - Call that the gradient

Partial Derivative

$$L_{CE}(w, b) = -[y \log \sigma(w \cdot x + b) + (1 - y) \log (1 - \sigma(w \cdot x + b))]$$

$$\frac{\partial L_{CE}(w, b)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

If we have n weights, we end up with n partial derivatives. The vector consisting of those derivatives is called the gradient. We'll use the gradient to update all the weights.

CE Loss Partial Derivative

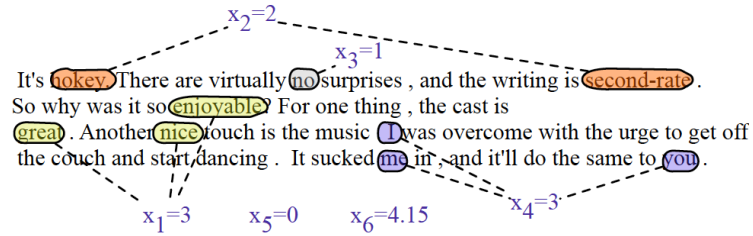
- Looks like gobbledygook, but it matches our earlier intuition about the loss and “credit/blame” assignment for the error

$$\frac{\partial L_{CE}(w, b)}{\partial w_j} = [\underbrace{\sigma(w \cdot x + b)}_{\text{Computed answer}} - \underbrace{y}_{\text{right answer}}] x_j$$

Take the difference

Multiply by the value of the input feature for the weight being adjusted

Scoring



[3,2,1,3,0,4.15]

$$\begin{aligned} p(+|x) &= P(Y = 1|x) = \sigma(w \cdot x + b) \\ &= \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.15] + 0.1) \\ &= \sigma(.805) \\ &= 0.69 \end{aligned} \tag{5.6}$$

$$\begin{aligned} p(-|x) &= P(Y = 0|x) = 1 - \sigma(w \cdot x + b) \\ &= 0.31 \end{aligned}$$

Example

- Current weight vector
 - $[2.5, -5, -1.2, 0.5, 2.0, 0.7]$
- Input vector (x)
 - $[3, 2, 1, 3, 0, 4.15]$
- Computed answer is: 0.69
- Correct answer is 1.

Example

$$\frac{\partial L_{CE}(w, b)}{\partial w_j} = [\underbrace{\sigma(w \cdot x + b) - y}_{-0.31}]x_j$$

Input: [3, 2, 1, 3, 0, 4.15]

Gradient:

[-0.31 * 3, -0.31 * 2, -0.31 * 1, -0.31 * 3, -0.31*0, -0.31*4.15]

=[-0.93, -0.62, -0.31, -0.93, 0, -1.28]

Update the Weights

- Weights_t:
 - [2.5, -5, -1.2, 0.5, 2.0, 0.7]
- Gradient:

$$w_{t+1} = w_t - \mu \Delta$$

= [-0.93, -0.62, -0.31, -0.93, 0, -1.28]

- Assume learning rate of 1 for simplicity
- Weights_t+1:
 - [3.42687096, -4.38208602, -0.89104301, 1.42687096, 2. , 1.9821715]

Do it Again

- Input vector (same)
 - [3, 2, 1, 3, 0, 4.15]
- New weights
 - [2.92, -4.72, -1.06, 0.92, 2.0, 1.281]
- Score
 - $\sigma([2.92, -4.72, -1.06, 0.92, 2.0, 1.281] \cdot [3, 2, 1, 3, 0, 4.15] + .1)$
 - = $\sigma(6.43)$
 - = .998

SGD

function STOCHASTIC GRADIENT DESCENT($L()$, $f()$, x , y) **returns** θ

where: L is the loss function

f is a function parameterized by θ

x is the set of training inputs $x^{(1)}, x^{(2)}, \dots, x^{(n)}$

y is the set of training outputs (labels) $y^{(1)}, y^{(2)}, \dots, y^{(n)}$

$\theta \leftarrow 0$

repeat T times

For each training tuple $(x^{(i)}, y^{(i)})$ (in random order)

Compute $\hat{y}^{(i)} = f(x^{(i)}; \theta)$ # What is our estimated output $\hat{y}$?

Compute the loss $L(\hat{y}^{(i)}, y^{(i)})$ # How far off is $\hat{y}^{(i)}$ from the true output $y^{(i)}$?

$g \leftarrow \nabla_{\theta} L(f(x^{(i)}; \theta), y^{(i)})$ # How should we move θ to maximize loss ?

$\theta \leftarrow \theta - \eta g$ # go the other way instead

return θ

Better Training

- Batch training
 - Process each example in the training set and accumulate the gradients
 - Do a single update
 - Repeat
- Minibatch training
 - Select N examples and proceed as with batch training
 - Update after each mini-batch
 - N is chosen to maximize parallelism
- Adjust the learning rate during training.

Multiclass Models

- While useful, binary classification isn't the typical use case in NLP. Rather we're classifying an object into one of N categories.
- We'd like a model that provides a probability distribution over the categories given an object.

Multiclass Models

- Multinomial logistic regression (Softmax)
- We'll still assume we have objects represented as vectors of features, and weights on the features, and a scoring function.

Softmax

$$p(y = c|x) = \frac{e^{w_c \cdot x + b_c}}{\sum_{j=1}^k e^{w_j \cdot x + b_j}}$$

Softmax Classification

- Given an object x , the softmax function gives us the probability distribution over the classes.
- We can take the argmax and go with that or we can simply use the distribution as the answer (passing it on for further processing).

Cross Entropy Loss

$$\begin{aligned} L_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) &= -\sum_{k=1}^K \mathbf{y}_k \log \hat{\mathbf{y}}_k \\ &= -\log \hat{\mathbf{y}}_c, \quad (\text{where } c \text{ is the correct class}) \\ &= -\log \hat{p}(\mathbf{y}_c = 1 | \mathbf{x}) \quad (\text{where } c \text{ is the correct class}) \\ &= -\log \frac{\exp(\mathbf{w}_c \cdot \mathbf{x} + b_c)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \quad (c \text{ is the correct class}) \end{aligned}$$

Cross Entropy Loss

$$L_{\text{CE}}(\hat{\mathbf{y}}, \mathbf{y}) = - \sum_{k=1}^K \mathbf{y}_k \log \hat{\mathbf{y}}_k$$

$$= -\log \hat{\mathbf{y}}_c, \quad (\text{where } c \text{ is the correct class})$$

$$= -\log \hat{p}(\mathbf{y}_c = 1 | \mathbf{x}) \quad (\text{where } c \text{ is the correct class})$$

$$= -\log \frac{\exp(\mathbf{w}_c \cdot \mathbf{x} + b_c)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \quad (c \text{ is the correct class})$$

CE Loss Partial Derivative

Looks like gobbledygook, but it matches our earlier intuition about the loss and “credit/blame” assignment for the error

$$\begin{aligned}\frac{\partial L_{\text{CE}}}{\partial \mathbf{w}_{k,i}} &= -(\mathbf{y}_k - \hat{\mathbf{y}}_k) \mathbf{x}_i \\ &= -(\mathbf{y}_k - p(\mathbf{y}_k = 1 | \mathbf{x})) \mathbf{x}_i \\ &= -\left(\underbrace{\mathbf{y}_k}_{\text{Correct answer}} - \underbrace{\frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)}}_{\text{Computed answer}} \right) \mathbf{x}_i\end{aligned}$$

Difference

Scaled by the magnitude of
the feature value.

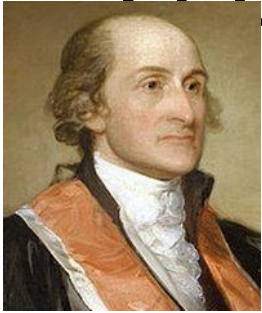
Partial Derivative

$$\begin{aligned}\frac{\partial L_{\text{CE}}}{\partial \mathbf{w}_{k,i}} &= -(\mathbf{y}_k - \hat{\mathbf{y}}_k) \mathbf{x}_i \\ &= -(\mathbf{y}_k - p(\mathbf{y}_k = 1 | \mathbf{x})) \mathbf{x}_i \\ &= -\left(\mathbf{y}_k - \frac{\exp(\mathbf{w}_k \cdot \mathbf{x} + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)} \right) \mathbf{x}_i\end{aligned}$$

If we have k weights, we end up with k partial derivatives. The vector consisting of those derivatives is called the gradient. We'll use the gradient to update all the weights.

$$p(y = c|x) = \frac{e^{w_c \cdot x + b_c}}{\sum_{j=1}^k e^{w_j \cdot x + b_j}}$$

Federalist Example

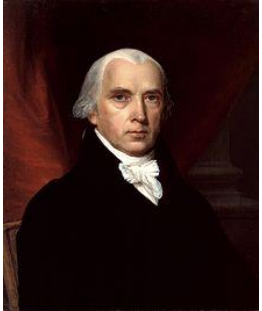


Scores

3 20 $P(\text{Jay} | \text{doc}) = 20/77.3 = .258$

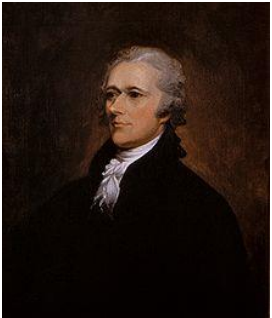
Gold

0



1 2.71 $P(\text{Madison} | \text{doc}) = 2.71/77.31 = .035$

0



4 54.6 $P(\text{Hamilton} | \text{doc}) = 54.6/77.31 = .706$

1

Learning Intuition

- Single Federalist example... the system said
[0.258, 0.035, 0.706]

And it should have said

[0, 0, 1]

So... move the feature weights in the direction where the component scores are closer to

[0, 0, 1].

Evaluation

- How do we know how good our systems are?
- Think about the possible outcomes for an email spam detector

2-by-2 Contingency Table

gold standard labels

		gold positive	gold negative	
<i>system output labels</i>	system positive	true positive	false positive	precision = $\frac{tp}{tp+fp}$
	system negative	false negative	true negative	
		recall = $\frac{tp}{tp+fn}$		accuracy = $\frac{tp+tn}{tp+fp+tn+fn}$

		Predicted condition			
		Positive (PP)	Negative (PN)	Informedness, bookmaker informedness (BM) $= \text{TPR} + \text{TNR} - 1$	Prevalence threshold (PT) $= \frac{\sqrt{\text{TPR} \times \text{FPR}} - \text{FPR}}{\text{TPR} - \text{FPR}}$
Actual condition	Positive (P)	True positive (TP), hit	False negative (FN), type II error, miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{\text{TP}}{\text{P}} = 1 - \text{FNR}$	False negative rate (FNR), miss rate $= \frac{\text{FN}}{\text{P}} = 1 - \text{TPR}$
	Negative (N)	False positive (FP), type I error, false alarm, overestimation	True negative (TN), correct rejection	False positive rate (FPR), probability of false alarm, fall-out $= \frac{\text{FP}}{\text{N}} = 1 - \text{TNR}$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{\text{TN}}{\text{N}} = 1 - \text{FPR}$
Prevalence $= \frac{\text{P}}{\text{P} + \text{N}}$		Positive predictive value (PPV), precision $= \frac{\text{TP}}{\text{PP}} = 1 - \text{FDR}$	False omission rate (FOR) $= \frac{\text{FN}}{\text{PN}} = 1 - \text{NPV}$	Positive likelihood ratio (LR+) $= \frac{\text{TPR}}{\text{FPR}}$	Negative likelihood ratio (LR-) $= \frac{\text{FNR}}{\text{TNR}}$
Accuracy (ACC) $= \frac{\text{TP} + \text{TN}}{\text{P} + \text{N}}$		False discovery rate (FDR) $= \frac{\text{FP}}{\text{PP}} = 1 - \text{PPV}$	Negative predictive value (NPV) $= \frac{\text{TN}}{\text{PN}}$ $= 1 - \text{FOR}$	Markedness (MK), deltaP (Δp) $= \text{PPV} + \text{NPV} - 1$	Diagnostic odds ratio (DOR) $= \frac{\text{LR}+}{\text{LR}-}$
Balanced accuracy (BA) $= \frac{\text{TPR} + \text{TNR}}{2}$		F ₁ score $= \frac{2\text{PPV} \times \text{TPR}}{\text{PPV} + \text{TPR}} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$	Fowlkes–Mallows index (FM) $= \sqrt{\text{PPV} \times \text{TPR}}$	Matthews correlation coefficient (MCC) $= \sqrt{\text{TPR} \times \text{TNR} \times \text{PPV} \times \text{NPV}} - \sqrt{\text{FNR} \times \text{FPR} \times \text{FOR} \times \text{FDR}}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{\text{TP}}{\text{TP} + \text{FN} + \text{FP}}$

Multinomial Problems

- What about when we have more than two classes?

		<i>gold labels</i>			
		urgent	normal	spam	
<i>system output</i>	urgent	8	10	1	precision_u = $\frac{8}{8+10+1}$
	normal	5	60	50	precision_n = $\frac{60}{5+60+50}$
	spam	3	30	200	precision_s = $\frac{200}{3+30+200}$
		recall_u = $\frac{8}{8+5+3}$	recall_n = $\frac{60}{10+60+30}$	recall_s = $\frac{200}{1+50+200}$	

A Combined Measure: F

- A combined measure that assesses the P/R tradeoff is F measure (weighted harmonic mean):

- People usually use balanced F₁ measure
 - i.e., with $\beta = 1$ (that is, $\alpha = 1/\beta$):

$$F_1 = \frac{2PR}{P + R}$$

Micro- vs. Macro-Averaging

- If we have more than one class, how do we combine multiple performance measures into one quantity?
 - Macro-averaging: Compute performance for each class, then average.
 - Micro-averaging: Collect decisions for all classes, compute contingency table, evaluate.

Micro- vs. Macro-Averaging: Example

Class 1

	Truth: yes	Truth: no
Classifier: yes	10	10
Classifier: no	10	970

Class 2

	Truth: yes	Truth: no
Classifier: yes	90	10
Classifier: no	10	890

Micro Ave. Table

	Truth: yes	Truth: no
Classifier: yes	100	20
Classifier: no	20	1860

- Macroaveraged precision: $(0.5 + 0.9)/2 = 0.7$
- Microaveraged precision: $100/120 = .83$
- Microaveraged score is dominated by score on common classes